

PRÁCTICA 3

Sincronización de procesos con mutex: Uso de mutex con 3 productores y 1 consumidor

ÍNDICE

Introducción	1
Códigos	1
Conclusiones	2

INTRODUCCIÓN

Para el apartado 3 de la práctica, se pide modificar el código del apartado 2 de resolución el problema del productor-consumidor empleando *mutex* y variables de condición para el caso en el que existan tres productores diferentes.

CÓDIGOS

Se trabajará con 1 único programa, que es el siguiente:

`prod_cons_prioridad.c` - Código del problema del productor-consumidor con 3 productores y resuelto con *mutex* y variables de condición.

COMPILACIÓN - `gcc -o prod_cons_prioridad prod_cons_prioridad.c -pthread`

EJECUCIÓN - `./prod_cons_prioridad`

Es necesario que en el mismo directorio del ejecutable existan 3 archivos de texto con los nombres `archivo1.txt`, `archivo2.txt`, y `archivo3.txt`, los cuales contengan números enteros separados por espacios.

FUNCIONAMIENTO - El código de este apartado es completamente análogo al del apartado 2. Las únicas diferencias con respecto a este último son:

- Hay 3 hilos para productores, numerados del 1 al 3. Cada uno leerá enteros del archivo de índice correspondiente a su identificador.
- En lugar de enteros, se trabajará con estructuras con 2 campos, un *item* y su prioridad, la cual será asignada en función del archivo del que se leyó el entero.
- Las variables de sumas de enteros se triplican, resultando en 3 pares de contadores de productor y de consumidor. Cada una de estas se actualiza si la prioridad del *item* se corresponde a su índice.
- Ahora hay 3 variables de condición, 1 para cada uno de los productores. Los productores solo pueden emplear su variable de condición y la del consumidor, mientras que el consumidor

despertará a uno de los productores dependiendo de la prioridad del *item* leído en la iteración actual del bucle.

- La función `remove_item()` hace 3 búsquedas de elementos, desde la prioridad 1 hasta la 3, y, en el momento en el que se encuentre un entero para eliminar, se sale de dicha función sin recorrer los lazos posteriores.

CONCLUSIONES

Al ejecutar el programa con 3 archivos de ejemplo, con los contenidos "1 2 3 4 5", "1 2 3 4" y "1 2 3 4 6", respectivamente, se obtienen los siguientes resultados:

- El orden en el que los productores generan *items* es aleatorio. Simplemente avanzará el primero de los hilos que obtenga el *mutex* y lo bloquee.

- A pesar del punto anterior, el consumidor respeta las prioridades de elementos del *buffer*. Siempre que se insertan varios enteros por parte de diferentes productores, el consumidor prioriza los de preferencia numéricamente más baja.

```
[PROD] Producido el item 2 con prioridad 1
[PROD] Producido el item 3 con prioridad 1
[PROD] Producido el item 3 con prioridad 3
[PROD] Producido el item 4 con prioridad 3
[PROD] Producido el item 4 con prioridad 1
      [CONS] Consumido el item 2 con prioridad 1
      [CONS] Consumido el item 3 con prioridad 1
```

Consumidor actuando en función de la preferencia de los items

- Al igual que en el apartado previo, no se producen condiciones de carrera, puesto que se puede apreciar cómo todas las variables de suma coinciden entre el productor y el consumidor correspondiente.

```
Suma [PROD1] = 15
Suma [CONS] de [PROD1] = 15

Suma [PROD2] = 10
Suma [CONS] de [PROD2] = 10

Suma [PROD3] = 16
Suma [CONS] de [PROD3] = 16
```

Contadores del consumidor coinciden con las sumas de los hilos pertinentes